

# 实验四：计费管理系统的 链表基本操作

# 本次实验授课内容

1. 动态内存分配与释放

2. 链表的概念及操作

- 什么是链表
- 建立简单的静态链表
- 建立动态链表
- 链表的基本操作：查找、插入

3. 实验四的内容与要求

# 什么是内存的动态分配

## 内存动态分配：

- 存放一些临时用的数据
  - 需要时随时开辟
  - 不需要时随时释放

## 为什么需要进行内存的动态分配？

- 结构体数组的长度是固定的（静态存储分配）
- 有些应用中结构体变量的数量无法预知，只能在程序运行中动态地向系统申请内存空间（动态存储分配）

# 怎样建立内存的动态分配

## 对内存的动态分配

- 通过系统提供的库函数来实现（主要有4个函数）
  - malloc, calloc, free, realloc
  - 以上函数的声明在stdlib.h头文件中
    - 在用到这些函数时应当用“#include <stdlib.h>”指令把stdlib.h头文件包含到程序文件中。

## malloc函数

- 函数原型:

**void \*malloc (unsigned int size);**

- 在内存的动态存储区中分配一个长度为size的连续空间
- 此函数的基类型为指针型函数void，即不指向任何类型的数据，只提供一个地址，返回的指针指向该分配区域的开头位置，即函数的值在所分配区域的第一个字节的地址
- **不成功**（例如内存空间不足），则返回空指针(**NULL**)

例如：

**malloc(100);**

- 开辟100字节的临时分配域
- 函数值为其第1个字节的地址

**malloc(sizeof(int));**

- 开辟长度为sizeof(int)的空间



## calloc函数

- 函数原型:

**void \*calloc(unsigned int n,unsigned int size);**

- 在内存的动态存储区中分配n个长度为size的连续空间
  - 用**calloc**函数可以为一维数组开辟动态存储空间（动态数组），n为数组元素个数，每个元素长度为size
  - 函数返回指向所分配域的起始位置的指针
  - 如果分配不成功，返回**NULL**

例如：

```
p = calloc (50,4);
```

开辟 $50 \times 4$ 个字节的临时分配区域  
把起始地址赋给指针变量p



## free函数

- 其函数原型为

```
void free(void *p);
```

- 其作用是释放指针变量 p 所指向的动态空间，使这部分空间能重新被其他变量使用
- p是最近一次调用calloc或malloc函数时得到的函数返回值
- free函数无返回值

例如：

**free(p);**

- 释放指针变量 **p** 所指向的已分配的动态空间

## realloc函数

- 其函数原型为

**void \*realloc (void \*p, unsigned int size);**

- 将p所指向的动态空间的大小改变为size。

p的值不变

- 如果重分配不成功，返回**NULL**。
- 如果已经通过**malloc**函数或**calloc**函数获得了动态空间，想改变其大小，可以用**realloc**函数重新分配。

例如：

```
realloc (p,50);
```

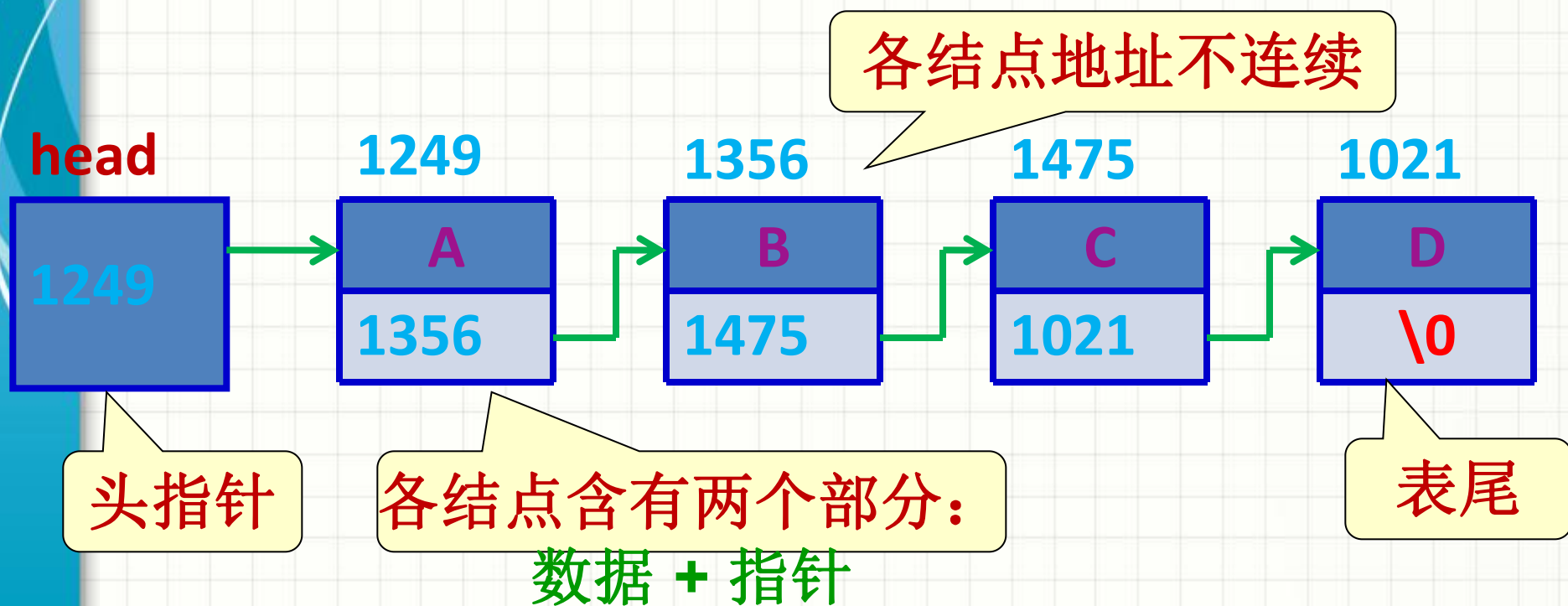
将**p**所指向的已分配的动态空间改为**50**字节

# 链表

什么是链表：

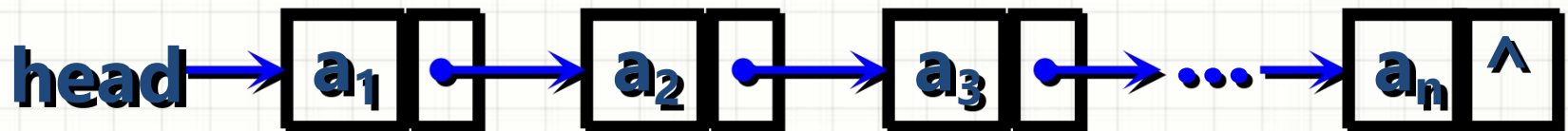
- 链表是一种常见的重要的数据结构
- 若干个数据项（称为“结点”）按一定的原则连接起来
- 每个结点都是一个结构体变量：
  - 若干数据 + 指针（指向下一个结点）
    - 链表必须利用指针变量才能实现
    - 依靠这些指针将所有的结点连接成一个链表
- 它是动态地进行存储分配的一种结构

# 链表示例

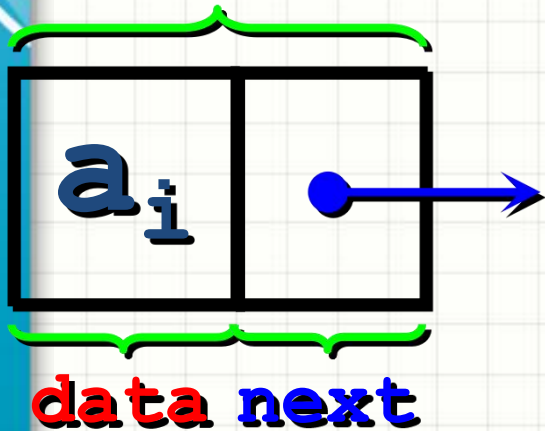




# 链表示例

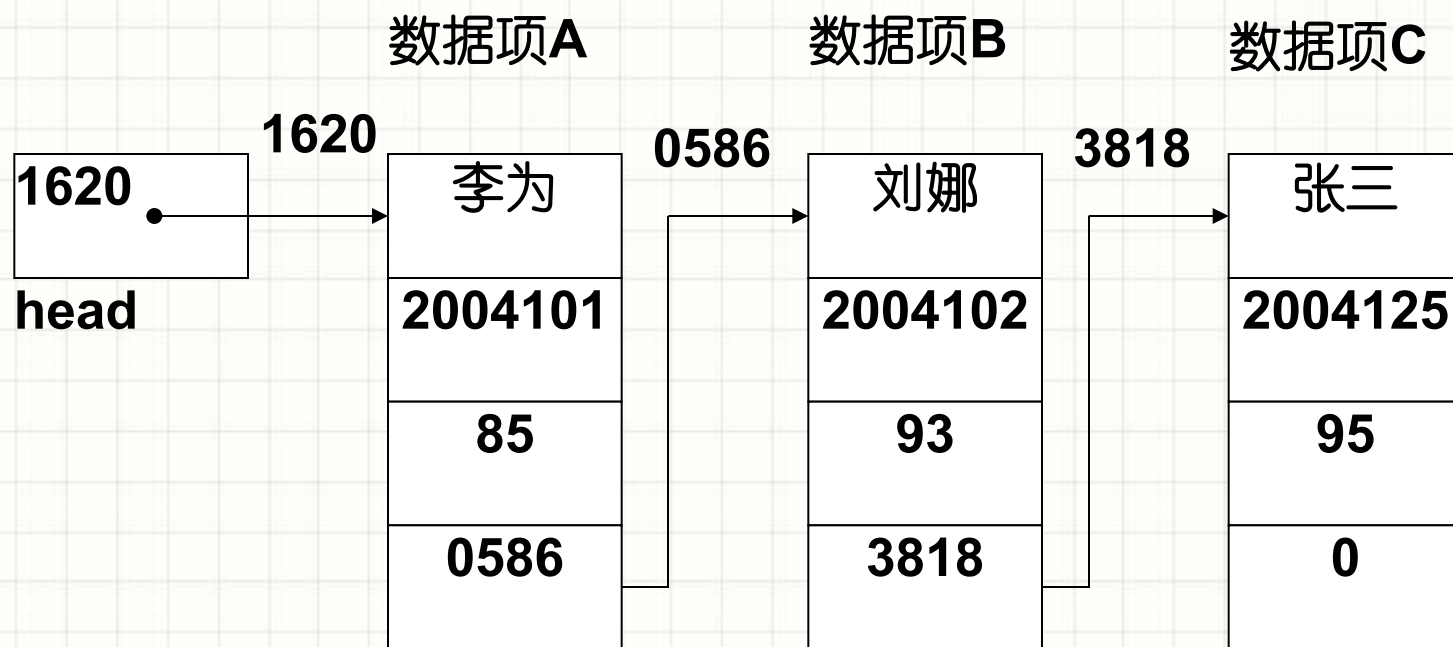


**struct student**



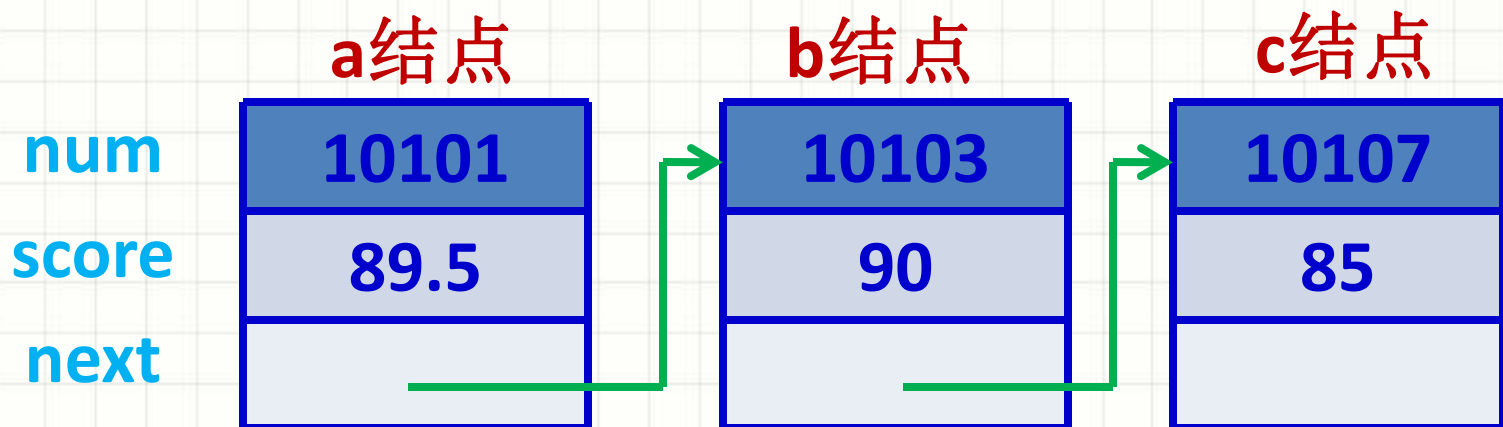
```
struct student {  
    char name[20];  
    long num;  
    float score;  
    struct student *next  
};  
struct student *head;
```

# 链表示例



# 建立简单的静态链表

例： 建立一个如图所示的简单链表，它由3个学生数据的结点组成，要求输出各结点中的数据。



# 建立简单的静态链表

```
struct Student
{
    int num;
    float score;
    struct Student *next;
} *head, a, b, c;
```

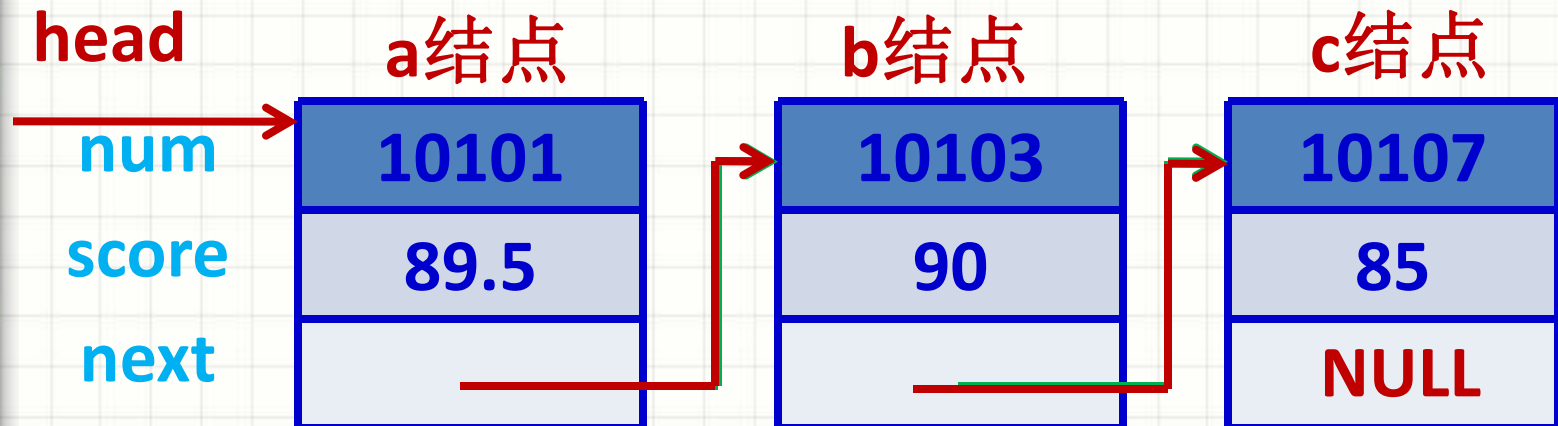
解题思路:

head=&a;

a.next=&b;

b.next=&c;

c.next=NULL;



```
#include <stdio.h>
struct Student
{ int num;
  float score;
  struct Student *next;
};
```

```
int main()
```

```
{ struct Student a,b,c,*head,*p;
```

```
  a.num=10101; a.score=89.5;
```

```
  b.num=10103; b.score=90;
```

```
  c.num=10107; c.score=85;
```

```
  head=&a;      a.next=&b;
```

```
  b.next=&c;      c.next=NULL;
```

```
  p=head;
```

```
  do
```

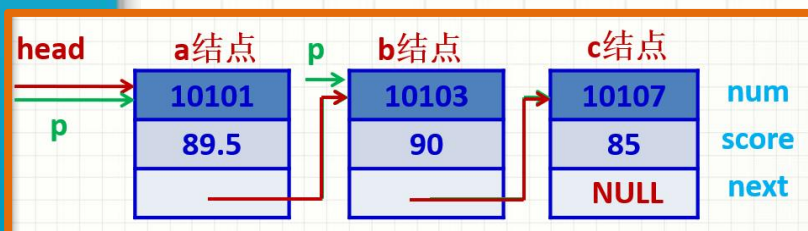
```
  {printf("%ld%5.1f\n",p->num, p->score);
```

```
    p=p->next;
```

```
  }while(p!=NULL);
```

```
  return 0;
```

```
}
```



```
10101 89.5
```

# 建立动态链表

所谓建立动态链表是指：

- 在程序执行过程中从无到有地建立起一个链表
- 逐个地开辟结点和输入各结点数据，并建立起前后相链的关系



**【示例】**： 写一函数建立一个有n名学生数据的单向动态链表

**【解题思路】**：

- 定义3个指针变量：**head**, **p1**和**p2**，它们都是用来指向struct Student类型数据 **struct Student \*head,\*p1,\*p2;**
- 用**malloc**函数开辟第一个结点，并使**p1**和**p2**指向它
- 读入一个学生的数据给**p1**所指的第一个结点
- 使**head**也指向新开辟的结点
- 再开辟另一个结点并使**p1**指向它，接着输入该结点的数据
- 使第一个结点的**next**成员指向第二个结点，即连接第一个结点与第二个结点
- 使**p2**指向刚才建立的结点
- 再开辟另一个结点并使**p1**指向它。。。

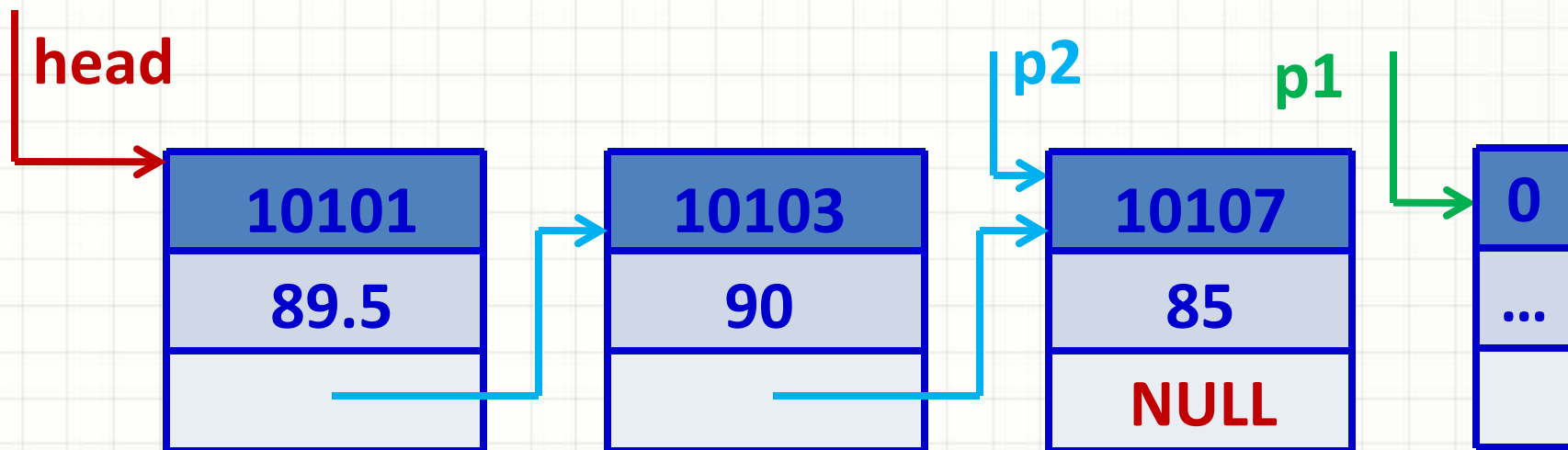
**p1**总是开辟新结点  
**p2**总是指向最后结点  
用**p2**和**p1**连接两个结点

## 【解题思路】：

**p1**总是开辟**新**结点

**p2**总是指向**最后**结点

用**p2**和**p1**连接两个结点



**struct Student**类型数据的长度

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define LEN sizeof(struct Student)
```

```
struct Student
```

```
{ long num;
```

```
  float score;
```

```
  struct Student *next;
```

```
};
```

```
int n;
```

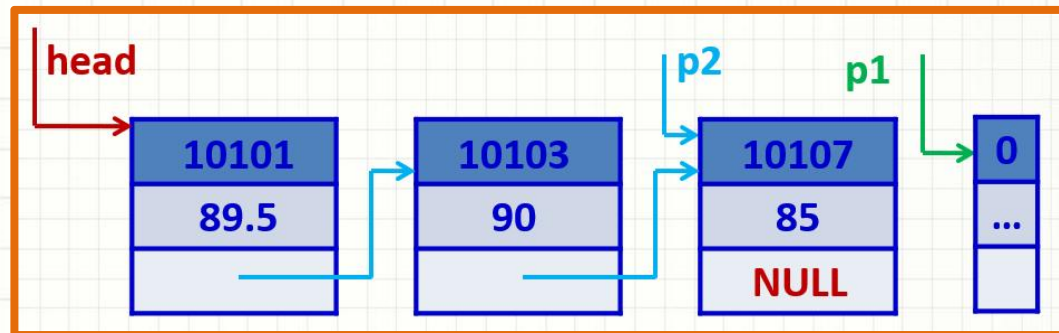
```

struct Student *creat(void)
{ struct Student *head,*p1,*p2; n=0;
  p1=p2=( struct Student*) malloc(LEN);

  scanf("%ld,%f",&p1->num,&p1->score);
  head=NULL;
  while(p1->num!=0)
  { n=n+1;
    if(n==1) head=p1;
    else p2->next=p1;
    p2=p1;
    p1=(struct Student*)malloc(LEN);
    scanf("%ld,%f",&p1->num,&p1->score);
  }
  p2->next=NULL;
  return(head);
}

```

**p1**总是开辟新结点  
**p2**总是指向最后结点  
 用**p2**和**p1**连接两个结点



# 输出链表

【示例】 编写一个输出链表的函数print()

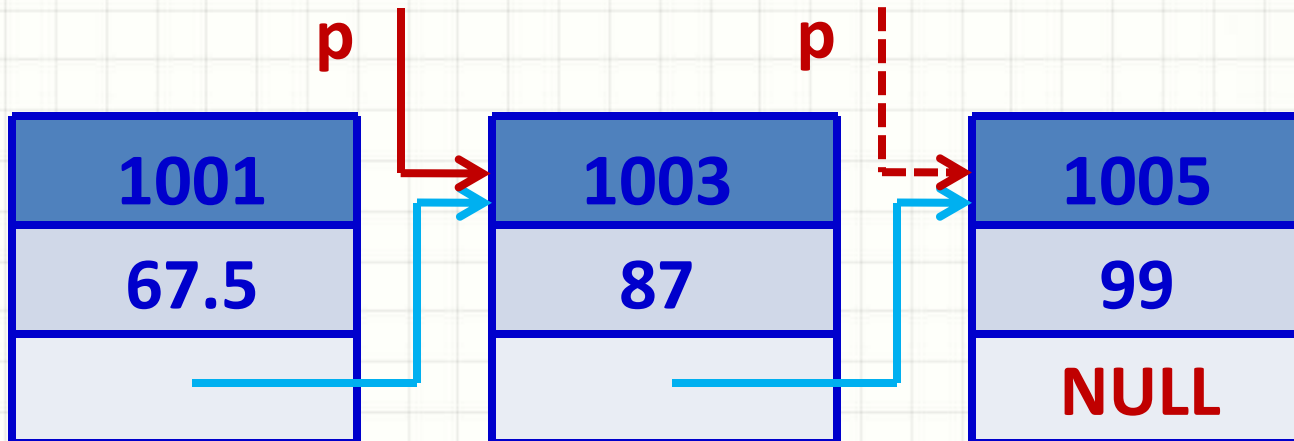
【解题思路】：

◆输出p所指的结点

```
printf("%ld %5.1f\n",p->num,p->score);
```

◆使p后移一个结点

```
p=p->next;
```



|      |      |
|------|------|
| 1001 | 67.5 |
| 1003 | 87.0 |
| 1005 | 99.0 |

```
void print(struct Student *p)
{
    printf("\nThese %d records are:\n",n);
    if(p!=NULL)
        do
        { printf("%ld %5.1f\n",
                p->num,p->score);
          p=p->next;
        }while(p!=NULL);
}
```

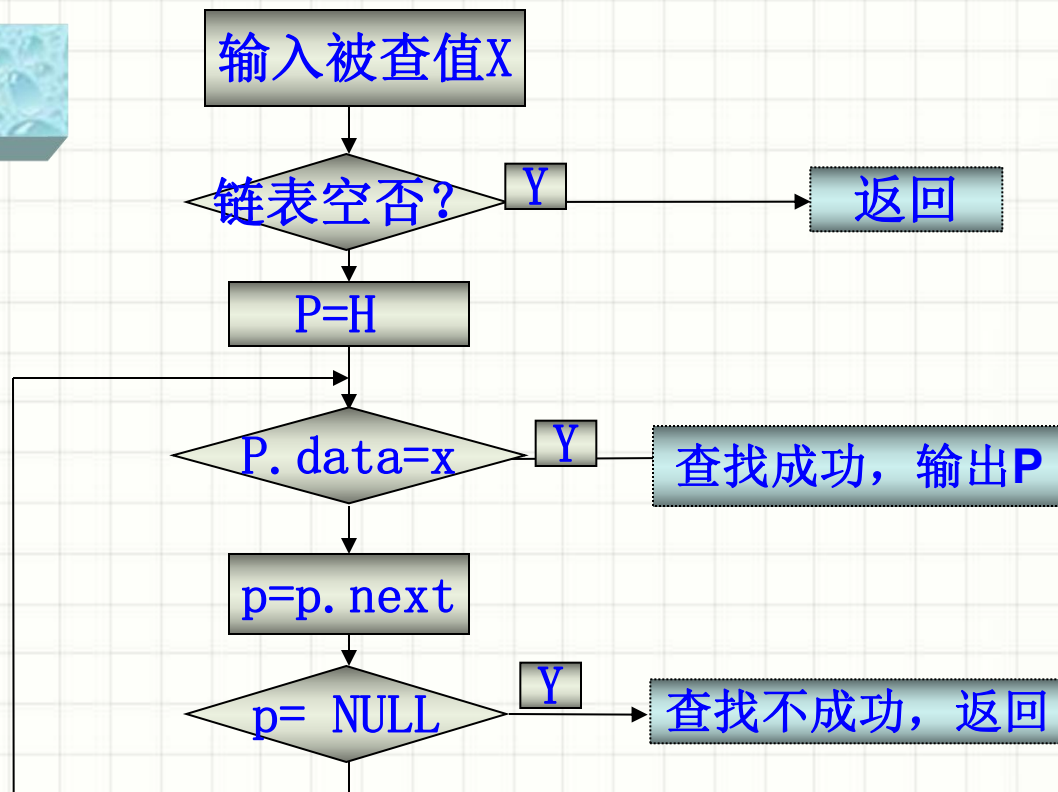


# 链表的查找操作

## 功能：

在链表中查找某结点，  
若存在则返回结点的地址，  
否则给出不存在的信息。

## 算法框图:



**方法:** 首先看链表空否?

不空则从表头结点的地址开始查找, 找到结束;  
找不到再取下一个结点的地址, 看其空否?  
若空则给出不存在的信息, 否则继续查找。

# 实验四： 计费管理系统的链表基本操作

## 实验任务：

- 1) 动态内存分配与释放
- 2) 定义链表结构，建立链表
- 3) 进行链表的插入、删除、查询、输出等基本操作
- 4) 将卡信息保存在链表，

实现卡管理： 添加卡、查询卡（链表）

# 实验四：计费管理系统的链表基本操作

## 实现卡管理：添加卡、查询卡（链表）

（保留前面实验的代码，构造新函数编码实现）

| 项目名称 | 实验内容            | 交付物         | 平台电子书相关章节                                     |
|------|-----------------|-------------|-----------------------------------------------|
| 第4次  | 计费管理系统的链表基本操作   | 运行截图 + 代码截图 | 《计费管理系统(C语言)》<br>第十二章 编码实现与测试阶段<br>VI：卡管理(链表) |
| 第5次  | 计费管理系统的数据动态存储管理 |             | 《计费管理系统(C语言)》<br>第十二章 编码实现与测试阶段<br>VI：卡管理(链表) |

**测试用例：** 分别输入菜单编号1、2；进而  
添加卡（密码正常/超长/重复卡号）；查询卡（存在/不存在）

**验收要求：** 本次实验的正常运行需包含上述测试用例